

Sr. No	Problem Statements
1	Design and implement a Student Course Management System database using SQL. Create tables such as: Student (rollno, name, address, phno, email_id, marks) , Courses (c_id, cname, rollno) using CREATE, ALTER, RENAME, TRUNCATE and DROP tables where required. Also, apply constraints: PRIMARY KEY on student and course IDs ,FOREIGN KEY in Course table, NOT NULL for essential fields (e.g., student name) , UNIQUE for email and CHECK for valid marks (0–100) . Insert at least 7 to 10 records into each table. Demonstrate constraint enforcement by attempting invalid insertions. Use ON DELETE CASCADE
	OR
1	Design and Implement following query using MongoDB 1. Create a collection called „games, 2. Add 5 games to the database. Give each document the following properties: name, gametype, rating (out of 100), 3. Write a query that returns all the games, 4. Write a query that returns the 3 highest rated games, 5. Update your two favourite games to have two achievements called „Game Master“ and Speed Demon“, 6. Write a query that returns all the games that have both the „Game Maser“ and 7. the „Speed Demon“ achievements. 8. Write a query that returns only games that have achievements.
2	Create a table Employee with attributes emp_id, emp_name, department, and salary. Insert at least 8–10 records into the table. Write a PL/SQL program using a TRIGGER to ensure that no employee can be inserted with a salary less than 10,000, and use a CURSOR to display all employee details along with the total salary of all employees.
	OR
2	Design and Implement following query using MongoDB Create a collection called „games, 2. Add 5 games to the database. Give each document the following properties: name, gametype, rating (out of 100), 3. Write a query that returns all the games, 4. Write a query that returns the 3 highest rated games, 5. Update your two favourite games to have two achievements called „Game Master“ and Speed Demon“, 6. Write a query that returns all the games that have both the „Game Maser“ and 7. the „Speed Demon“ achievements. 8. Write a query that returns only games that have achievements.

3	Create a table Employees (emp_id, emp_name, salary, dept_id) and Departments (dept_id, dept_name). Use Appropriate Keys 1. Insert at least 8–10 records in the Employees table and 3–5 records in Departments. 2. Write a query to display employees who earn more than the average salary. 3. Write a query to find the employee(s) with the highest salary. 4. Write a query to display employees whose salary is greater than the average salary of their department (correlated subquery). 5. Write a query to display employees working in departments where the maximum salary is greater than 60,000. 6. Write a query using NOT IN to find employees who are not assigned to any department. 7. Write a query using EXISTS to display employees who belong to departments located in a specific condition.
	OR
3	Create a table Employee with attributes emp_id, emp_name, department, and salary. Insert at least 8–10 records into the table. Write a PL/SQL program using a TRIGGER to ensure that no employee can be inserted with a salary less than 10,000, and use a CURSOR to display all employee details along with the total salary of all employees.
4	Named PL/SQL Block: PL/SQL Stored Procedure and Stored Function. Write a Stored Procedure namely proc_Grade for the categorization of student. If marks scored by students in examination is ≤ 1500 and $\text{marks} \geq 990$ then student will be placed in distinction category if marks scored are between 989 and 900 category is first class, if marks 899 and 825 category is Higher Second Class. Write a PL/SQL block to use procedure created with above requirement. Stud_Marks(name, total_marks) Result(Roll, Name, Class)
	OR
4	Design and implement a Student Course Management System database using SQL. Create tables such as: Student (rollno, name, address, phno, email_id, marks) , Courses (c_id, cname, rollno) using CREATE, ALTER, RENAME, TRUNCATE and DROP tables where required. Also, apply constraints: PRIMARY KEY on student and course IDs ,FOREIGN KEY in Course table, NOT NULL for essential fields (e.g., student name) , UNIQUE for email and CHECK for valid marks (0–100) . Insert at least 7 to 10 records into each table. Demonstrate constraint enforcement by attempting invalid insertions. Use ON DELETE CASCADE

5	Develop a database for an Online Shopping System using SQL. Create tables such as: Customer (c_id, name, address, phno, email) ,Product (p_id, pname, c_id, price, qty). Using CREATE tables , ALTER to modify structure, RENAME any one table , TRUNCATE and DROP operations for testing . Also apply constraints: PRIMARY KEY for each table , FOREIGN KEY in product referencing Customers , NOT NULL for important fields (name, price) , UNIQUE for customer email or product code , CHECK for price > 0 and quantity ≥ 1 Insert sample data (minimum 7 to 10 records per table), Test integrity constraints by inserting incorrect data.ON DELETE CASCADE.
	OR
5	Design and implement a Banking Management System database to manage customer accounts and transactions.Create tables such as: Customers (cust_id, name, balance, city) Transactions (txn_id, cust_id, amount, txn_type) Perform: INSERT records for customers and transactions , UPDATE account balance after deposits/withdrawals , DELETE inactive customers . Use: Arithmetic operations (balance + amount) , Logical operators (AND, OR) , Pattern matching (LIKE 'A%') , String functions (UPPER(), LENGTH()) , Set operators (UNION to combine customer lists).
6	A MongoDB collection named orders contains documents with the following structure: Orders(Id, name, product, category, amt, date) 1) Find the total sales amount for each category. 2) Find the average order amount per category. 3) ☐ Sort categories by total sales in descending order.
	OR
6	Create the following tables. Solve queries by SQL Deposit (actno,cname,bname,amount,adate) Branch (bname,city) Customers (cname, city) Borrower(loanno,cname,bname, amount) Add primary key and foreign key wherever applicable. Insert data into the above created tables. Solve following queries by SQL 1. Display customer name having living city Bombay and branch city Nagpur 2. Display customer name having same living city as their branch city 3. Display customer name who are borrowers as well as depositors and having living city Nagpur. 4. Display borrower names having deposit amount greater than 1000 and loan amount greater than 2000 5. Display customer name living in the city where branch of depositor sunil is located. 6. Create an index on deposit table.

7	Create a database : 1. employee (employee name, street, city) ,employee name is primary key 1. works (employee name, company name, salary), 3. company (company name, city) ,company name is primary key, 4. manages (employee name, manager name), Give an expression in SQL for each of the following queries. 1. Find the names of all employees who work for First Bank Corporation. 2. Find all employees who do not work for First Bank Corporation 3. Find the company that has most employees. 4. Find all companies located in every in which small bank corporation is located 5. Find details of employee having salary greater than 10,000. 6. Update salary of all employees who work for First Bank Corporation by 10%. 7. Find employee and their managers. 8. Find the names, street and cities of all employees who work for First Bank Corporation And earn more than 10,000. 9. Find those companies whose employees earn a higher salary,on average, than the average salary at First Bank Corporation.
	OR
7	A MongoDB collection named orders contains documents with the following structure: Orders(Id, name, product, category, amt, date) 1) Find the total sales amount for each category. 2) Find the average order amount per category. 3) Sort categories by total sales in descending order.
8	Improve Query Performance using Indexing, Consider a collection users: (Id, name, mailed, age, city) 1. Create an index on the email field to improve search performance. 2. Create a compound index on city and age. 3. Query users from Mumbai aged above 20 efficiently.
	OR
8	Create a database for a Student Course Management System. • Create the following tables with appropriate attributes and constraints: ○ Students ○ Courses ○ Enrollments • Insert at least 5 records in each table. Write SQL queries to: 1 Display student names, course names, and marks using INNER JOIN. 2 Display all students including those not enrolled in any course using LEFT JOIN. 3 Display all courses including those with no enrolled students using RIGHT JOIN. 4 Display all possible student–course combinations including unmatched records using FULL OUTER JOIN (or equivalent). 5 Create a view named Student_Course_View to display Student Name, Course Name, and Marks. 6 Create another view to display students who have scored more than 75 marks. 7 Retrieve data from both created views. 8 Drop any one of the created views.

9	Stored Function with Cursor for Aggregation Logic Create a reusable function to: (Use suitable data) 1. Iterate through orders 2. Calculate total sales per customer 3. Store results in another collection customer_total
	OR
9	SQL Queries for Data Manipulation, Access Control, and Transactions Design and run SQL queries to demonstrate the following: Data Manipulation (DML): Use SQL statements to INSERT, UPDATE, and DELETE records. Apply arithmetic, logical, set operators, pattern matching, and string functions.
10	Stored Function with Cursor for Aggregation Logic Create a reusable function to: (Use suitable data) 1. Iterate through orders 2. Calculate total sales per customer Store results in another collection customer_total
	OR
10	Develop a Library Management System database to manage books and members. Create tables: a. Members (member_id, name, email) , Books (book_id, title, author, price) b. Borrow (borrow_id, member_id, book_id, date) , Perform: INSERT records into all tables , UPDATE book price or member details , DELETE records for returned/lost books. Use:Arithmetic operations (price calculation with discount) , Logical conditions (price > 500 AND author='XYZ') , Pattern matching (title LIKE '%SQL%') , String functions (LOWER(), CONCAT()) , Set operators (INTERSECT or UNION for reports).

11	Develop a Library Management System database to manage books and members. Create tables: Members (member_id, name, email) , Books (book_id, title, author, price), Borrow (borrow_id, member_id, book_id, date) ,Create roles: librarian and assistant use: GRANT full access to librarian , GRANT limited access to assistant (SELECT only) , REVOKE permissions where needed. Apply TCL commands: START TRANSACTION, Issue a book (insert into Borrow + update stock) , Create SAVEPOINT before updating stock , ROLLBACK if book not available , COMMIT after successful issue
	OR
11	Develop a database to analyze sales performance of a company. Create a table: Sales (sale_id, product_name, region, quantity, price) Insert at least 10–15 records covering multiple products and regions. Write SQL queries to: 1. Calculate total sales amount (SUM(quantity * price)) for each product. 2. Find average sales per region using AVG(). 3. Identify highest and lowest sales using MAX() and MIN(). 4. Use GROUP BY to group data by product_name and region. 5. Use HAVING to: ▪ Display products with total sales > 5000 ▪ Show regions where average sales < 1000
12	Design and demonstrate transaction handling in a Banking System using TCL commands. Consider the following Scenario: A customer wants to transfer money from one account to another. The transaction must ensure data consistency. Tasks:Start a transaction using START TRANSACTION, Deduct amount from Sender's account, Create a SAVEPOINT after deduction, Credit the same amount to Receiver's account, Simulate an error condition (e.g., insufficient balance or system failure), Use ROLLBACK to revert to the savepoint if an error occurs. If all operations are successful, use COMMIT to permanently save changes.
	OR
12	CRUD Operations using MongoDB Design and implement basic Create, Read, Update, and Delete (CRUD) operations using MongoDB. Use the save method and logical operators wherever necessary.

13	Design a database to analyze student academic performance using aggregate functions. Create a table: - Results (student_id, student_name, subject, marks) - Insert at least 10–15 records with multiple subjects per student. Write SQL queries to: - Calculate total marks and average marks of each student using aggregate functions (SUM(), AVG()). - Display the maximum and minimum marks scored in each subject. - Use GROUP BY to group data by student_id and subject. - Use HAVING to: - Show students with average marks greater than 70 - Display subjects where maximum marks > 90
	OR
13	CRUD Operations using MongoDB Design and implement basic Create, Read, Update, and Delete (CRUD) operations using MongoDB. Use the save method and logical operators wherever necessary.
14	Solve Below Queries: Use Set operations, Tuple Variable and Order by clause - Find all customers who have a loan, an account, or both: - Find all customers who have both a loan and an account. - Find all customers who have an account but no loan. - Find the customer names and their loan numbers and amount for all customers having a loan at some branch using tuple variable - Find the names of all branches that have greater assets than some branch located in Brooklyn using tuple variable - List in alphabetic order the names of all customers having a loan in Perryridge branch Use Depositor and borrower relation with appropriate data” Create table and insert 5 rows.
	OR
14	Write a named PL/SQL block using a Stored Procedure to categorize students based on their total marks. Consider the following tables: Stud_Marks (name, total_marks) Result (roll, name, class) Create a stored procedure named proc_Grade to assign a class to students based on their marks: - Marks 990 to 1500 → <i>Distinction</i> - Marks 900 to 989 → <i>First Class</i> - Marks 825 to 899 → <i>Higher Second Class</i> - Create the Stored Procedure (proc_Grade) to categorize students - Insert the categorized data into the Result table - Write a PL/SQL block to call the procedure

15	Write SQL queries to: 1 Display student names, course names, and marks using INNER JOIN. 2 Display all students including those not enrolled in any course using LEFT JOIN. 3 Display all courses including those with no enrolled students using RIGHT JOIN. 4 Display all possible student–course combinations including unmatched records using FULL OUTER JOIN (or equivalent). 5 Create a view named Student_Course_View to display Student Name, Course Name, and Marks. 6 Create another view to display students who have scored more than 75 marks. 7 Retrieve data from both created views. 8 Drop any one of the created views.
	OR
15	Database Triggers Implement and test triggers to maintain data integrity in database Use appropriate data and implement Before and after
16	Create a db called company consist of the following tables. 1. Emp (eno,ename, job,hiredate,salary,commission,deptno,) 2.dept(deptno,deptname,location) eno is primary key in emp deptno is primary key in dept Solve Queries by SQL 1. List the maximum salary paid to salesman 2. List name of emp whose name start with „I“ 3. List the emp details in the descending order of their basic salary 4. List the avg salary, minimum salary of the emp hire datewise for dept no „10“. 5. List emp name and its department 6. List total salary paid to each department 7. List details of employee working in „Dev“ department.
	OR
16	Write a database trigger on the Library table. The system should keep track of records that are updated or deleted. The old values of updated or deleted records must be stored in the Library_Audit table. Implement triggers covering: • Row-level triggers • Statement-level triggers • BEFORE triggers • AFTER triggers The Library_Audit table should store the previous values of the records whenever an update or delete operation is performed on the Library table.

